

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «Docker Refresher»

Dauer: 100 Minuten

Besprechung: 20 Minuten

Total: 120 Minuten

In diesem Refresher wiederholen Sie Docker, Docker Compose und Docker Images praktisch. Ziel ist, dass die wichtigsten Befehle, Begriffe und Fehlerbilder wieder sitzen, bevor der Kurs zu Kubernetes wechselt.

Offizielle Dokumentation

<https://docs.docker.com/>

<https://docs.docker.com/compose/>

<https://docs.docker.com/build/building/best-practices/>

Aufgabe 1)

Docker Mental Model

Ordnen Sie zuerst die wichtigsten Begriffe. Arbeiten Sie bewusst kurz und präzise.

a) Erklären Sie die Begriffe Image, Container, Dockerfile, Registry, Compose und Kubernetes in je einem Satz.

Image = Vorlage für Container, unveränderlich
Container = laufende Instanz vom Image
Dockerfile = Bauanleitung fürs Image
Registry = Server wo die Images liegen, z.B. Docker Hub
Compose startet mehrere Container mit einer YAML
Kubernetes betreibt und skaliert Container auf mehreren Rechnern

b) Beschreiben Sie den Weg von einem Image zu einem laufenden Container bis zur Fehlersuche mit Logs.

```
docker pull nginx
docker run -d --name web -p 8080:80 nginx
docker ps
docker logs web
```

c) Nennen Sie zwei Dinge, die Container erleichtern, und zwei Dinge, die Container nicht automatisch lösen.

+ läuft überall gleich, Services sind isoliert
- ohne Volume sind Daten weg, Monitoring und Skalierung muss man selber machen

Aufgabe 2)

Container-Check mit NGINX

Starten, untersuchen und entfernen Sie einen einzelnen Container sauber.

- a) Starten Sie einen NGINX-Container im Hintergrund mit dem Namen «refresh-web» und Port-Mapping 8080:80. Notieren Sie den Befehl.**

Tipp: docker run, -d, --name, -p

```
docker run -d --name refresh-web -p 8080:80 nginx
```

- b) Prüfen Sie Status, Container-ID und Port-Mapping. Welche Ausgabe ist für Sie relevant?**

Tipp: docker ps / docker inspect

```
docker ps
Status Up und 0.0.0.0:8080->80/tcp, dazu die Container-ID
```

- c) Öffnen Sie die Webseite im Browser. Welche URL verwenden Sie und woran erkennen Sie, dass der Container erreichbar ist?**

```
localhost:8080, es kommt Welcome to nginx!
```

- d) Öffnen Sie eine Shell im Container und schauen Sie sich das Verzeichnis der ausgelieferten Webseite an.**

Tipp: docker exec -it refresh-web sh

```
docker exec -it refresh-web sh
ls /usr/share/nginx/html
index.html und 50x.html
```

- e) Versuchen Sie einen zweiten Container auf demselben Host-Port zu starten. Was passiert und wie lösen Sie das Problem sauber?**

```
docker run -d --name refresh-web2 -p 8080:80 nginx
port is already allocated, 8080 ist schon besetzt
anderen Port nehmen z.B. 8081:80 oder den ersten stoppen
```

- f) Stoppen und löschen Sie alle Container aus dieser Aufgabe.**

```
docker rm -f refresh-web refresh-web2
```

Aufgabe 3)

Compose Mini-Stack

Beschreiben Sie eine kleine Umgebung mit zwei Services als Compose-Datei. Aus diesem Content

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
  redis:
    image: redis:alpine
```

a) Starten Sie die Umgebung. Welche Container, Netzwerke und Ports entstehen?

Tipp: docker compose up -d / docker compose ps / docker network ls

```
docker compose up -d
web-1 und redis-1, Netz docker-refresh-lab_default
nur web hat Port 8080 nach aussen, redis nur intern
```

b) Zeigen Sie Logs für die gesamte Umgebung und danach nur für den Web-Service an. Was ist der Unterschied?

```
docker compose logs
docker compose logs web
erst web und redis zusammen, dann nur web
```

c) Prüfen Sie, ob Redis antwortet. Was bedeutet die Antwort «PONG»?

Tipp: docker compose exec redis redis-cli PING

```
docker compose exec redis redis-cli PING
PONG, Redis läuft und antwortet
```

d) Ergänzen Sie ein benanntes Volume für Redis. Warum ist ein benanntes Volume etwas anderes als Daten im Container-Dateisystem?

```
redis:
  volumes:
    - redis_data:/data
volumes:
  redis_data:
  Volume bleibt auch wenn der Container gelöscht wird
```

e) Stoppen Sie die Umgebung einmal ohne und einmal mit Volume-Löschung. Welche Befehle verwenden Sie?

Tipp: docker compose down / docker compose down -v

```
docker compose down
docker compose down -v
```

Aufgabe 4)

Eigenes Image bauen

Erstellen Sie eine sehr kleine eigene Webseite und verpacken Sie diese als Image.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
LABEL org.opencontainers.image.title="docker-refresh-web"
```

a) Erstellen Sie eine einfache «index.html». Welche sichtbare Information soll Ihre Seite enthalten?

```
Titel und mein Name, damit man sieht dass es mein Image ist
<h1>Docker Refresher</h1>
<p>Kilian Hirschi - TEK0 - Tag 1</p>
```

b) Bauen Sie das Image mit dem Tag «docker-refresh-web:1.0.0». Notieren Sie den Befehl und die wichtigste Build-Ausgabe.

```
docker build -t docker-refresh-web:1.0.0 .
naming to docker.io/library/docker-refresh-web:1.0.0
```

c) Starten Sie einen Container aus Ihrem eigenen Image und prüfen Sie die Webseite im Browser.

```
docker run -d --name myweb -p 8081:80 docker-refresh-web:1.0.0
localhost:8081 zeigt meine Seite
```

d) Erstellen Sie eine «.dockerignore». Welche Dateien oder Ordner gehören nicht in den Build Context?

```
.git, .env, node_modules, *.log
braucht das Image nicht, in .env sind Secrets
```

e) Ändern Sie nur die «index.html» und bauen Sie erneut. Welche Layer werden wiederverwendet und warum?

Tipp: docker history / Build-Cache-Ausgabe

FROM ist CACHED, nur der COPY Schritt wird neu gebaut weil die Datei anders ist

Aufgabe 5)

Tags, Debugging und Registry-Readiness

Bereiten Sie Ihr Image so vor, dass es später eindeutig referenziert werden könnte. Ein Push in eine Registry ist hier nicht nötig.

a) Taggen Sie Ihr Image zusätzlich als «teko/docker-refresh-web:1.0.0» und «teko/docker-refresh-web:latest». Was ist der Unterschied zwischen diesen Tags?

docker tag docker-refresh-web:1.0.0 teko/docker-refresh-web:1.0.0
docker tag docker-refresh-web:1.0.0 teko/docker-refresh-web:latest
zeigen aufs gleiche Image, 1.0.0 ist fix, latest kann wandern und ist nicht unbedingt das Neuste

b) Welche Informationen finden Sie mit «docker inspect» bei Image und Container, die für spätere Deployments wichtig sind?

Image: Port 80, Cmd, Architektur arm64
Container: IP, Ports, Mounts, Status

c) Fehlerbild 1: «port is already allocated». Welche Befehle verwenden Sie zur Diagnose und wie beheben Sie das Problem?

docker ps, docker port refresh-web, lsof -i :8080
Container stoppen oder anderen Port nehmen

d) Fehlerbild 2: «pull access denied» oder «image not found». Welche Ursachen sind wahrscheinlich?

Tippfehler im Namen oder Tag, privates Repo ohne docker login, falsche Registry

e) Notieren Sie Ihre persönliche Docker-Debugging-Checkliste mit mindestens sechs Befehlen.

docker ps -a
docker logs
docker inspect
docker exec -it ... sh
docker compose logs -f
docker images
docker port